

Projet IA – IRVE

Stations de Recharge pour Véhicules Électriques · FISE3 2026

Contexte

Application d'analyse de la base nationale IRVE (Infrastructures de Recharge pour Véhicules Électriques). Le projet couvre clustering, classification et régression avec scikit-learn, ainsi que de la visualisation cartographique.

Les 4 Besoins Clients

BC1 — Carte visuelle

Personne 1

« Montre-moi où sont les bornes sur une carte »

- Charger le CSV, garder lat/lon + type d'implantation
- Créer une carte interactive avec les bornes colorées par type (Folium)
- Créer une heatmap qui montre les zones denses en bornes
- Pas de modèle IA ici — uniquement de la visualisation

Livrables : [notebook .ipynb](#) | [carte_implantation.html](#) | [heatmap_densite.html](#)

BC2 — Clustering géographique

Personne 2

« Regroupe les bornes par zones géographiques automatiquement »

- Prendre lat/lon et appliquer KMeans (ou DBSCAN)
- Tester K de 2 à 12 et choisir le meilleur via la courbe Elbow
- Évaluer avec 3 métriques : Silhouette, Calinski-Harabasz, Davies-Bouldin
- Sauvegarder le modèle .pkl + écrire le script CLI

Livrables : [notebook .ipynb](#) | [predict_cluster.py](#) | [model_clustering.pkl](#)

BC3 — Classification implantation

Personne 1

« Prédit le type d'implantation d'une borne (voirie, parking...) »

- Classification — cible : `implantation_station`
- Choisir un algo (Random Forest conseillé), optimiser via GridSearchCV
- Métriques : accuracy, F1-score, matrice de confusion
- Sauvegarder le modèle .pkl + écrire le script CLI

Livrables : [notebook .ipynb](#) | [predict_implantation.py](#) | [model_classification.pkl](#)

BC4 — Régression puissance nominale

Personne 2

« Prédit la puissance en kW d'une borne (7kW ? 22kW ? 50kW ?) »

- Régression — cible : puissance_nominale (valeur numérique continue)
- Même logique que BC3 mais avec RandomForestRegressor
- Métriques : MAE (erreur moyenne), RMSE, R^2 (variance expliquée)
- Sauvegarder le modèle .pkl + écrire le script CLI

Livrables : [notebook .ipynb](#) | [predict_puissance.py](#) | [model_regression.pkl](#)

Points communs à tous les besoins

Notebook .ipynb	Toute l'expérimentation commentée, exécutable sans erreur
Script .py CLI	Prend des arguments, charge le .pkl, retourne une prédiction — ne réentraîne JAMAIS
Justifications	Pourquoi ces variables ? Pourquoi cet algo ? → dans le rapport (pas dans le code)
Modèles .pkl	Sauvegarder avec joblib.dump() — inclure dans l'archive finale

Livrable final & Évaluation

Archive ZIP	projetia_trinomeX.zip · 4 dossiers Besoin_Client_N + rapport.pdf
Rapport	11 pages PDF · 1 page Gantt · AUCUN code · attendus justifiés par besoin
Soutenance	10 min strict + 5 min questions · démo live des 3 scripts CLI en 2 min
Barème	Soutenance 40 % — Compétences 40 % — Rapport + code 20 %

Diagramme de Gantt

8 juin → 12 juin 2026 · Binôme · 5 jours



Chaque case représente une demi-journée de travail · AM = matin, PM = après-midi